Le robot de réconfort

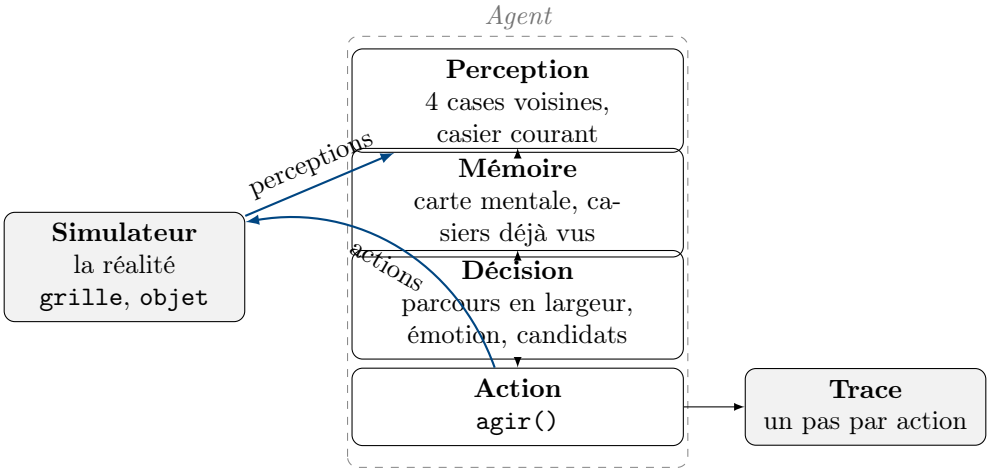
HAI716 – Programmation orientée agent et systèmes multi-agents

EL KAMEL Mohamed Amine • BAUDRAND Ethan

Master Informatique – Université de Montpellier – 2026–2027

Le robot doit apporter un objet de réconfort aux résidents d'un appartement. Pour chaque message, il lit l'émotion dans un dictionnaire, cherche l'objet correspondant dans une armoire de 24 casiers, puis va le donner au résident. Il ne connaît ni les murs ni le contenu des casiers : il voit seulement ses quatre cases voisines, et il ne voit un casier qu'en se plaçant devant l'armoire. Ce rapport présente notre architecture, notre façon de replanifier, notre avis sur la règle de repli et trois cas où notre robot se comporte mal.

1. Architecture



Le robot ne communique avec la réalité que par les perceptions et les actions.

Le simulateur est le seul module qui lit la grille et le contenu des casiers. Il répond aux perceptions, applique les actions et lève une exception si une action est illégale. Le robot reçoit seulement ce qu'il a le droit de savoir au départ (dimensions, positions de l'armoire, du dictionnaire et des résidents, étiquettes des casiers), et le reste vient de ses perceptions. Un test relit le code des autres modules et échoue si l'un d'eux mentionne `grille` ou `objet` : ce qui est interdit est vérifié absent à chaque exécution des tests.

À l'intérieur de l'agent, nous avons séparé la mémoire, la décision et l'action.

La mémoire ne retient que ce qui a été vu. Dans la carte mentale, chaque case est inconnue, libre ou occupée. Supposer qu'une case inconnue est libre n'est pas une information mais un choix de planification : cette hypothèse est donc dans `planification.py` et pas dans la carte mentale. C'est pareil pour l'armoire : `MemoireArmoire` retient ce qui a été constaté, `candidats` décide de l'ordre de fouille.

La décision ne connaît pas la mémoire. `plus_court_chemin` ne reçoit pas la carte, mais une fonction qui dit si une case est franchissable. On peut donc le tester sur une grille écrite à la main, et au projet 2 il suffira de changer cette fonction pour éviter les cases occupées par un autre robot.

Toutes les actions passent par `agir`, qui exécute l'action puis l'écrit dans la trace. Une action faite ailleurs manquerait dans la trace. `agir` relève aussi la position et le casier avant l'action, comme le demande le format.

Nos tests de bout en bout vérifient cette trace : ils rejouent chaque trace sur un simulateur neuf, et chaque position, perception et contenu de casier doit correspondre exactement à la réalité.

2. La stratégie de replanification

Nous recalculons le plan à chaque pas. Le robot perçoit ses quatre voisines, met à jour sa carte, calcule un plus court chemin en supposant libres les cases inconnues, puis avance d'une seule case. Quand un mur apparaît, il n'y a rien à détecter : le calcul suivant se fait sur une carte à jour et passe ailleurs.

Le coût est d'un parcours en largeur par pas, proportionnel au nombre de cases. Sur notre plus grande carte, 15×21 soit 315 cases, cela fait environ un millier de parcours pour un scénario complet , soit quelques millisecondes. Réparer le plan aurait coûté moins cher, mais il aurait fallu garder un état entre deux pas et traiter à part le cas où le mur coupe le chemin. Vu la taille des cartes, nous avons gardé la version simple.

Quand le parcours ne trouve aucun chemin, c'est une preuve. Le robot suppose libres toutes les cases qu'il n'a pas encore vues : il imagine donc l'appartement plus ouvert qu'il ne l'est. Si même ainsi aucun chemin n'existe, il n'en existe pas dans la réalité. C'est ainsi que nous détectons un résident inaccessible, sans explorer tout l'appartement.

Carte	Scénario	Demandes	Réussies	Échecs	Pas
appartement_01	scenario_01	4	4	0	210
appartement_02	scenario_02	5	5	0	259
appartement_03	scenario_03	6	6	0	311
appartement_04	scenario_04	6	6	0	300
appartement_05	scenario_05	8	8	0	328
appartement_06	scenario_06	9	8	1	1063
appartement_07	scenario_07	6	4	2	119
appartement_01	scenario_08	4	4	0	231

Les trois dernières lignes sont nos propres jeux de test ; leurs échecs sont voulus.

Les cinq jeux fournis passent tous. Dans les nôtres, un message n'a aucun mot reconnu (scénario 06) et un résident est muré puis sollicité deux fois (scénario 07). Le scénario 08 reprend l'appartement 01 mais avec une autre armoire, `armoire_colonne_vide`, dont plusieurs colonnes sont vides : il déclenche les quatre niveaux de repli (`aucun`, `intensite`, `voisine_1` et `voisine_2`), que les armoires fournies ne provoquent jamais toutes. Le coût baisse à mesure que la carte se remplit : sur le scénario 06, les premières demandes coûtent 161 et 147 pas, les dernières 80 et 114, pour des trajets comparables.

3. Critique de la règle de repli

L'idée de la règle est simple : sur la roue de Plutchik, deux émotions voisines se ressemblent, donc leurs objets devraient se ressembler aussi. Elle a deux avantages. Elle est déterministe, donc deux binômes corrects livrent le même objet. Et elle évite les replis les plus absurdes, puisque les distances 3 et 4 sont exclues.

Le problème c'est que ça confond deux choses : la proximité entre deux émotions, et le fait que leurs objets puissent se remplacer. Que la tristesse et le dégoût soient voisins ne dit rien sur l'effet d'un objet rangé sous « dégoût » sur une personne triste. Dans notre scénario 08, pour une tristesse

moyenne avec les casiers voisins vides, le robot livre l'objet rangé en colère moyenne, à deux crans sur la roue. Ça passe parce que l'objet est générique, mais la limite est que la règle regarde l'étiquette du casier, jamais ce que l'objet sert à faire.

Nous proposons de décider à partir de l'objet et non plus de l'émotion. Chaque casier indiquerait les émotions auxquelles son objet convient : une couverture pour la tristesse et la peur, un téléphone pour la tristesse et la peur. Le robot prendrait alors le casier le plus proche dont l'objet convient à l'émotion lue, et la roue ne servirait plus qu'à départager les casiers restants. La règle resterait déterministe, puisque tout serait écrit dans les fichiers.

Il faudrait pour cela ajouter un champ dans le fichier d'armoire : pour chaque casier, la liste des émotions acceptées. Le champ `repli` de la trace ne suffirait plus, car il ne sait décrire qu'une distance sur la roue ; il faudrait une valeur en plus pour dire que l'objet convient mais qu'il est rangé ailleurs. Enfin, la validation devrait vérifier que ces listes ne contiennent que des émotions connues, et qu'aucune émotion ne se retrouve sans aucun objet.

4. Trois échecs de notre robot

La négation n'est pas prise en compte. Nous nous arrêtons au premier mot reconnu, sans regarder les mots autour. « Je ne suis pas triste » est donc lu comme de la tristesse, et le robot apporte une couverture à quelqu'un qui va bien. C'est la règle de l'énoncé, que nous avons appliquée telle quelle. Pour corriger, il faudrait gérer les mots de négation, donc changer le format du dictionnaire.

Le sélecteur repasse sur ses pas. Nous traitons les casiers candidats un par un, et nous allons au plus court vers chacun sans regarder la suite de la liste. Quand une colonne est vide, le robot la descend, remonte, puis repart dans l'autre sens, en repassant devant des casiers qu'il sait déjà vides. Il faudrait planifier un itinéraire sur toute la liste au lieu de décider casier par casier.

Le robot fait un détour au premier contact. Il ne peut porter qu'un objet, et rien ne permet de le reposer. Notre première version prenait l'objet avant de savoir si le résident était joignable : avec un résident muré, le robot gardait l'objet en main et toutes les demandes suivantes échouaient. Ce défaut est apparu grâce à notre carte de test `appartement_07`, où un résident est muré. Nous allons maintenant voir le résident avant d'aller à l'armoire. Le problème est réglé, mais cela coûte un aller-retour la première fois : sur les cinq cartes fournies, le total passe de 1280 à 1408 pas. Nous mémorisons les résidents déjà atteints pour ne payer ce détour qu'une fois par résident.